COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 10, 2026	**DATE SUBMITTED:**	September 10, 2026
GROUP #:	**Group 5** (Ramos, Reyes, Rodulfa, Salcedo, Santiago)	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK 8,1 Using Sorting Algorithms 2

1. PROCEDURE OUTPUT

1. Explain the quick sort, shell sort and merge sort types of sorting algorithms.

- A quick sort is a sorting algorithm that uses a “divide-and-conquer” approach the same as in merge sort. The way it works is that it first chooses one element as a pivot, putting smaller elements on one side and larger elements on the other, then repeating the same process on both sides. For instance, we have 5, 3, 8, 2, 7. We choose 5 as the pivot. From there, smaller numbers go to the left, and large number will go to the right: 5, 2 | 5 | 8, 7. We then sort each group until we get 2, 3, 5, 7, 8. You would want to use quick sorting if you are presented with a big array/dataset of unsorted elements, hence the name “quick sort”.

- A shell sort is an efficient comparison-based sorting algorithm that acts as an optimization of standard insertion sort algorithm. It works by setting the gap size, performing gapped sorting, reducing the gap, and when the gap finally reaches 1, it iterates or passes through the entire array to ensure that it is perfectly sorted. Shell sorting is mostly used in devices with relatively small memory, such as embedded systems and microchips due to its use of nested for loops rather than recursion, ultimately eliminating the risk of stack overflows.

- A merge sort utilizes a “divide and conquer” sorting algorithm strategy by dividing the array (by finding its midpoint), creating a left and right section. These sections are also divided, usually into pairs. These pairs are then compared to each other, assorted into their desired order, and then merged with its accompanying pair. This process is done in both sections and uses the FOIL technique (First, Outer, Inner, and Last). After both the left and right section of subpairs have been sorted, the FOIL technique is

used once more to merge the 2 major sections together, which make up the target/sorted array. Because of its simple initialization and sorting operation, merge sorting is mostly used in Linked List-based programs and/or search filter systems.

2. Give simple sample programs in C++ that use the above sorting algorithms. Use a user input of 10 integer values in your example elements in an array to be sorted. Explain how the programs work.

Shell Sort

```cpp
#include <iostream>
int main() {
    int arr[10];
    std::cout << "Enter 10 integer values:\n";
    for (int i = 0; i < 10; i++) {
        std::cin >> arr[i];
    }
    // Shell Sort
    for (int gap = 5; gap > 0; gap /= 2) {
        for (int i = gap; i < 10; i++) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
    std::cout << "Sorted array:\n";
    for (int i = 0; i < 10; i++) {
        std::cout << arr[i] << " ";
    }
    return 0;
}
```

Code Analysis

The code works by asking the user for 10 integer values to create the elements for the array. The outer loop of the Shell Sort sets the gap for every iteration, starting with a gap of 5 and gradually reducing it until it reaches 1. The middle loop iterates through the array from left to right, starting at the index of the current gap. The while loop then compares the elements that are separated by the gap. If the element on the left is larger than the current element, it moves the larger value to the right and continues comparing until the correct position of the value is found. This process helps arrange the elements from the smallest value to the largest value. After the outer loop finishes and all the elements have been properly arranged, the last part of the code uses another loop to display the sorted array to the user.

Merge Sort

```cpp
#include <iostream>

void merge(int arr[], int left, int mid, int right) {
    int temp[10];
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j <= right) {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (int i = left; i <= right; i++) {
        arr[i] = temp[i];
    }
}
```

```cpp

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[10];

    std::cout << "Enter 10 integer values:\n";

    for (int i = 0; i < 10; i++) {
        std::cin >> arr[i];
    }

    mergeSort(arr, 0, 9);
    std::cout << "Sorted array:\n";

    for (int i = 0; i < 10; i++) {
        std::cout << arr[i] << " ";
    }
    return 0;
}
```

Code Analysis

The program begins by creating an array that can store 10 integer values. The user is asked to enter 10 numbers, and a for loop stores each number inside the array. After receiving the input, the program calls the mergeSort() function with 0 as the first position and 9 as the last position. This tells the program to sort the entire array.

The mergeSort() function works by dividing the array into smaller sections (divide and conquer). It finds the middle position using (left + right) / 2. The array is then divided into a left section and a right section. The function calls itself again to continue dividing these sections until they become very small (one element). Once the sections have been divided, the merge() function is used to put them back together in the correct order. The merge() function compares the values from the left and right sections and places the smaller value into a temporary array. It continues

	doing this until all the values have been arranged. The sorted values are then copied from the temporary array back into the original array. Finally, the program uses a for loop to display the sorted numbers. In simple terms, this works by splitting the array into smaller parts, sorting those parts, and then combining them back together in order.
Quick Sort	

```cpp
#include <iostream>

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);

        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}

```

```cpp
int main() {
    int arr[10];

    std::cout << "Enter 10 integer values:\n";

    for (int i = 0; i < 10; i++) {
        std::cin >> arr[i];
    }

    quickSort(arr, 0, 9);

    std::cout << "Sorted array:\n";

    for (int i = 0; i < 10; i++) {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```

Code Analysis

The program also starts by creating an integer array that can hold 10 values. The user enters 10 integer values, and the for loop stores them in the array. The program then calls the quickSort() function, starting from position 0 and ending at position 9. The Quick Sort algorithm uses a value called a pivot to help arrange the numbers. In this program, the last value of the section being sorted is chosen as the pivot.

The partition() function compares the other values with the pivot. If a value is smaller than the pivot, it is moved toward the left side of the array. Values that are larger than the pivot stay on the right side. The program uses a temp variable when swapping two values so that neither value is lost. After the comparison is finished, the pivot is placed in its correct position. The quickSort() function then sorts the values on the left side of the pivot and the values on the right side (divide and conquer). It continues this process until there are no more sections that need sorting. Finally, the program displays the sorted array using a for loop. In simple terms, it chooses a pivot, puts smaller values on one side and larger values on the other side, and then repeats the process until the array is sorted.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

N/A

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.